



CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E INGENIERÍAS



Parseo

Interacción Humano Computadora



Practica #6

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM

Profesor: Jovan Ricardo Zepeda Gómez

Fecha: 11/09/2026

Índice

Índice	1
Tabla de ilustraciones.....	1
Desarrollo.....	1
Interfaz.....	1
Clase.....	3
Bibliografía	5

Tabla de ilustraciones

Ilustración 1. Vista de la actividad principal.....	2
Ilustración 2. Vista calculadora.....	3
Ilustración 3. Clase "Opcion" – asignaciones.....	4
Ilustración 4. método "onClick".....	5

Desarrollo

La solicitud de esta actividad fue la de desarrollar una pequeña calculadora capaz de tomar los datos necesarios por medio de una interfaz con los elementos que anteriormente habíamos visto en clase, además de usar una nueva función llamada “parse” que se encarga de transformar un elemento tipo cadena de caracteres a un tipo operable (como enteros, flotantes o dobles).

La función requiere de estar dentro de un bloque de código que atrape interrupciones en el programa (try-catch), pues la función parse lanza excepciones si el objeto dentro no es solo un número.

Interfaz

Esta nueva función se encuentra dentro del menú principal, remplazando el botón de “Refresh” por *Calculadora*, que te llevará a la nueva vista con las opciones de la calculadora.

Parseo

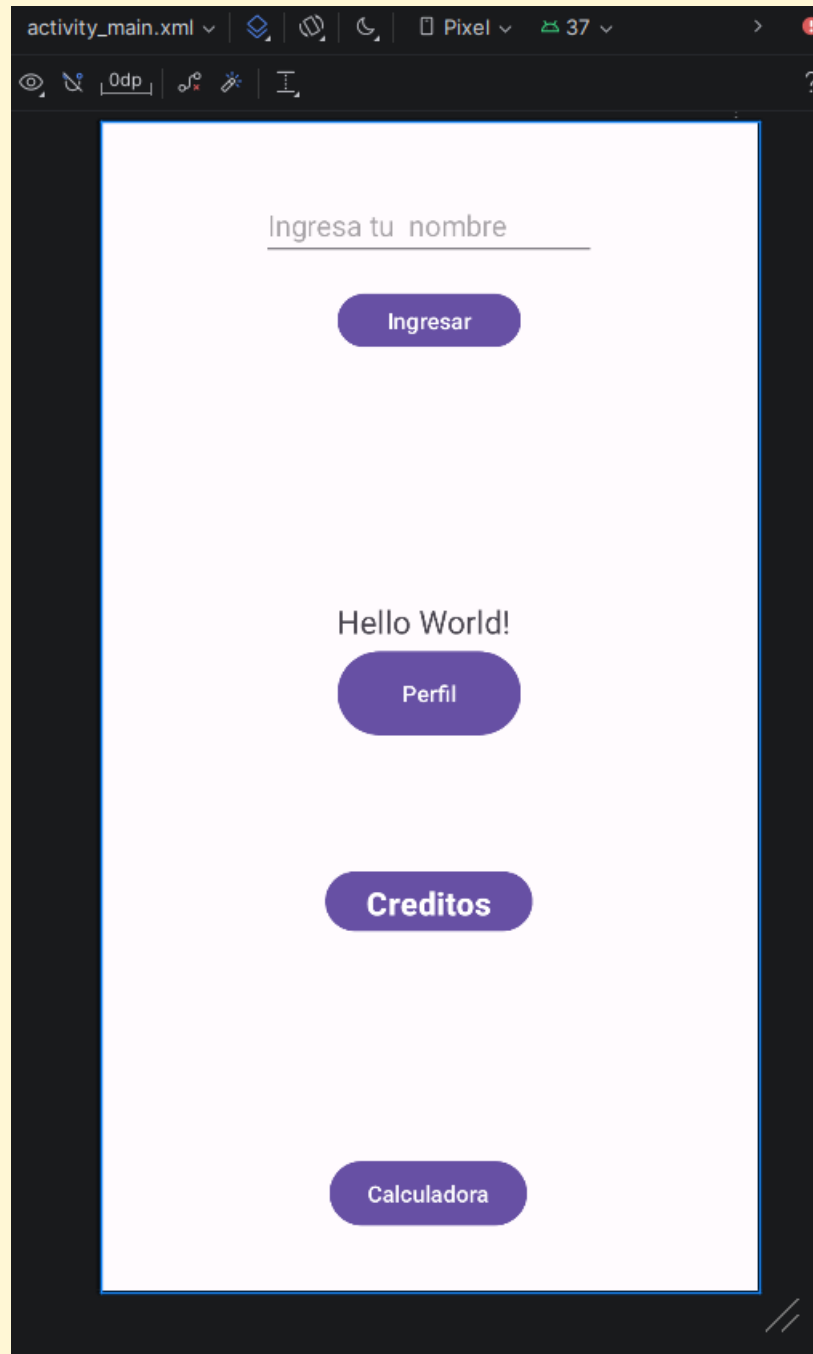


Ilustración 1. Vista de la actividad principal

En la vista de opción están las opciones para ingresar los dos valores necesarios, las distintas opciones para operar con ellos, una ventana por donde sale el resultado y un botón para limpiar los valores ingresados.

Para la alineación se utilizó un diseño vertical (vertical layout) para el espacio de ingresar datos y los botones de: suma, resta, multiplicación y división. Y para la parte del resultado se insertó un diseño libre (constraint layout) en el que se colocó una etiqueta para ingresar el resultado y debajo el botón para limpiar las entradas.

Interacción Humano Computadora

Para los nombres de las variables de elementos en pantalla se está utilizando la convención *Camel Case*, y todas especifican el tipo de elemento que son, por ejemplo: btnSumar, lblResultado, txtPrimerDato.

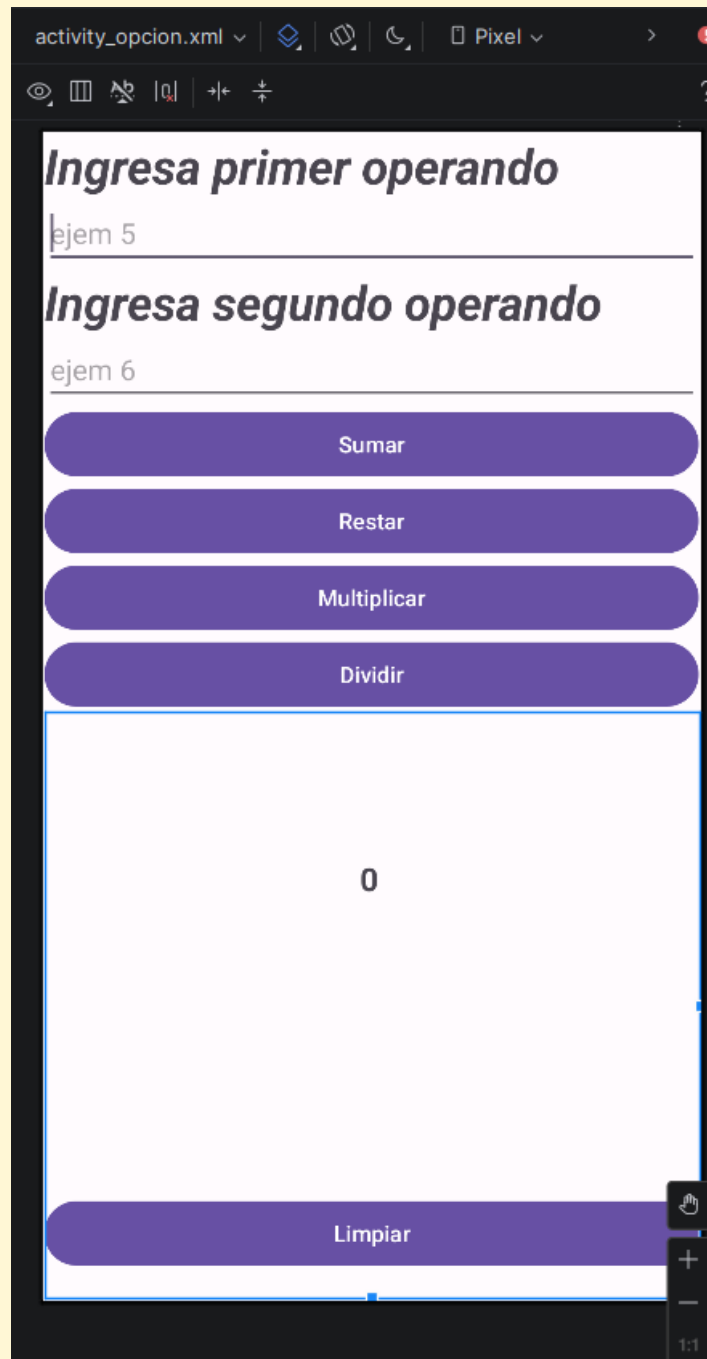


Ilustración 2. Vista calculadora

Clase

Dentro de su clase controladora correspondiente se encuentra la definición y recolección de todos los elementos de la vista opción que son necesarios, como lo

Parseo

puede ser: los elementos para editar texto, botones y la etiqueta de texto que muestra el resultado.

```
14 >< public class Opcion extends AppCompatActivity implements View.OnClickListener {
15
16     3 usages
17     private TextView txtRes;
18     3 usages
19     private EditText edt1, edt2;
20     2 usages
21     private Button btnSumar, btnRestar, btnMultiplicar, btnDividir, btnLimpiar;
22
23     @Override
24     protected void onCreate(Bundle savedInstanceState) {
25         super.onCreate(savedInstanceState);
26         setContentView(R.layout.activity_opcion);
27
28         txtRes = (TextView) findViewById(R.id.lblResultado);
29         edt1 = (EditText) findViewById(R.id.txtPrimerOperando);
30         edt2 = (EditText) findViewById(R.id.txtSegundoOperando);
31         btnSumar = (Button) findViewById(R.id.btnSumar);
32         btnRestar = (Button) findViewById(R.id.btnRestar);
33         btnMultiplicar = (Button) findViewById(R.id.btnMultiplicar);
34         btnDividir = (Button) findViewById(R.id.btnDividir);
35         btnLimpiar = (Button) findViewById(R.id.btnLimpiar);
36
37         btnSumar.setOnClickListener(this);
38         btnRestar.setOnClickListener(this);
39         btnMultiplicar.setOnClickListener(this);
40         btnDividir.setOnClickListener(this);
41         btnLimpiar.setOnClickListener(this);
42     }
43 }
```

Ilustración 3. Clase "Opcion" – asignaciones

La función *onClick* cuenta con una estructura más grande y está a cargo de la función de todos los botones. Y se ejecuta de la forma:

1. Verifica si el botón llamado fue *Limpiar*, si es así entonces limpia las entradas y la etiqueta del resultado para después volver.
2. Obtiene los valores de ambos operandos, los transforma en cadenas de caracteres y limpia espacios al inicio o final que puedan existir.
3. Verifica si alguna entrada está vacía, si alguna lo está, lanza un mensaje al usuario por medio de un *Toast*.
4. Una vez que validó las entradas, entra en el bloque sensible try-catch para hacer la conversión de valores de tipo carácter a entero. Si la función lanza una excepción de tipo formato de número, entonces el programa lanza un mensaje al usuario para indicarle que solo debe ingresar caracteres numéricos.
5. Al salir del bloque sensible, el código se dedica a verificar que operación se llamó a hacer, la ejecuta y guarda el resultado.
 - a. Si la operación fue división, entonces el programa realizará una verificación extra con el segundo operando. Si es cero, entonces lanzará un mensaje advirtiéndole al usuario de que la operación no se puede llevar a cabo y retorna, sino realiza la operación.

6. Para finalizar, el resultado se ingresa en la etiqueta de resultado al convertir el valor en tipo carácter.

```
41 @Override
42 public void onClick(View v) {
43     double x, y, res = 0;
44     String strX, strY;
45
46     if (v.getId() == R.id.btnLimpiar){
47         edt1.setText("");
48         edt2.setText("");
49         txtRes.setText("0");
50         return;
51     }
52
53     strX = edt1.getText().toString().trim();
54     strY = edt2.getText().toString().trim();
55
56     if (strX.isEmpty() || strY.isEmpty()) {
57         Toast.makeText(context, this, "Por favor llena ambos campos", Toast.LENGTH_SHORT).show();
58         return;
59     }
60
61     try {
62         x = Double.parseDouble(strX);
63         y = Double.parseDouble(strY);
64
65     } catch (NumberFormatException e){
66         Toast.makeText(context, this, "Solo caracteres numéricos", Toast.LENGTH_SHORT).show();
67         return;
68     }
69
70     if (v.getId() == R.id.btnSumar){
71         res = x + y;
72     } else if (v.getId() == R.id.btnRestar) {
73         res = x - y;
74     } else if (v.getId() == R.id.btnMultiplicar){
75         res = x * y;
76     } else if (v.getId() == R.id.btnDividir){
77         if (y == 0) {
78             Toast.makeText(context, this, "No se puede dividir entre cero", Toast.LENGTH_SHORT).show();
79             return;
80         }
81         res = x / y;
82     }
83
84     txtRes.setText(String.valueOf(res));
85 }
86
87 }
```

Ilustración 4. método "onClick"

Bibliografía

- Android API reference. (s/f). Recuperado el 11 de septiembre de 2026, de Android Developers website: <https://developer.android.com/reference>